

Primero haría un QA pequeño para comprobar que al restringir a la dirección principal ya no tenemos multiplicidad problemática:

```
padd_primary_qa <- sdf_sql(
  sc,
  "
  WITH positives AS (

    SELECT
      party_id,
      observation_date

    FROM bu_frd.model_features_ab_final_2022_2025

    WHERE label_3m = 1
  ),

  x AS (

    SELECT
      f.party_id,
      f.observation_date,

      COUNT(*) AS rows_per_party_date,
      COUNT(DISTINCT p.state_cde) AS distinct_states

    FROM positives f

    INNER JOIN bc3.padd p

      ON f.party_id = p.party_id
      AND p.data_date = f.observation_date
      AND p.addr_seq_num = 1

    WHERE p.data_date >= TO_DATE('2022-01-01')
      AND p.data_date < TO_DATE('2026-01-01')

    GROUP BY
      f.party_id,
      f.observation_date
  )

  SELECT
    COUNT(*) AS matched_party_dates,

    SUM(
      CASE WHEN rows_per_party_date > 1
```

```

        THEN 1 ELSE 0
    END
) AS duplicated_party_dates,

SUM(
    CASE WHEN distinct_states > 1
        THEN 1 ELSE 0
    END
) AS party_dates_multiple_states,

MAX(rows_per_party_date)
    AS max_rows_same_party_date,

MAX(distinct_states)
    AS max_states_same_party_date

FROM x
"
) %>%
collect()

padd_primary_qa

```

Idealmente queremos:

matched_party_dates	62599
duplicated_party_dates	0 (o muy bajo)
party_dates_multiple_states	0
max_rows_same_party_date	1
max_states_same_party_date	1

Si `duplicated_party_dates > 0` pero `party_dates_multiple_states = 0`, tampoco sería grave: probablemente son varias versiones de la misma dirección principal dentro del snapshot. Como tenemos `upd_tsp`, podemos quedarnos con la versión más reciente.

En ese caso usaría esta construcción definitiva:

```

positive_primary_state <- sdf_sql(
  sc,
  "
  WITH positives AS (

    SELECT
      party_id,
      observation_date,
      observation_year,
      label_3m

```

```
FROM bu_frd.model_features_ab_final_2022_2025

WHERE label_3m = 1
),
```

```
ranked_address AS (
```

```
SELECT
    f.party_id,
    f.observation_date,
    f.observation_year,
    f.label_3m,

    p.state_cde,
    p.city_name,
    p.data_date AS address_data_date,
    p.upd_tsp,
```

```
ROW_NUMBER() OVER (
```

```
    PARTITION BY
        f.party_id,
        f.observation_date
```

```
    ORDER BY
        p.upd_tsp DESC
```

```
) AS rn
```

```
FROM positives f
```

```
LEFT JOIN bc3.padd p
```

```
    ON f.party_id = p.party_id
    AND p.data_date = f.observation_date
    AND p.addr_seq_num = 1
```

```
    AND p.data_date >= TO_DATE('2022-01-01')
    AND p.data_date < TO_DATE('2026-01-01')
```

```
)
```

```
SELECT
    party_id,
    observation_date,
    observation_year,
    label_3m,
    state_cde,
```

```
city_name,  
address_data_date,  
upd_tsp
```

```
FROM ranked_address
```

```
WHERE rn = 1
```

```
"
```

```
) %>%
```

```
collect()
```

```
positive_primary_state
```

Aquí la regla queda muy clara:

Party → sameobservationdate → primaryaddress(addr_seq_num = 1) → latestupd_tsp

Eso es mucho más sólido que usar todos los addresses.

Después podemos calcular directamente la distribución por estado:

```
positive_state_distribution <- sdf_sql(
```

```
sc,
```

```
"
```

```
WITH positives AS (
```

```
SELECT
```

```
party_id,
```

```
observation_date
```

```
FROM bu_frd.model_features_ab_final_2022_2025
```

```
WHERE label_3m = 1
```

```
),
```

```
ranked_address AS (
```

```
SELECT
```

```
f.party_id,
```

```
f.observation_date,
```

```
p.state_cde,
```

```
ROW_NUMBER() OVER (
```

```
PARTITION BY
```

```
f.party_id,
```

```
f.observation_date
```

```

ORDER BY
    p.upd_tsp DESC

) AS rn

FROM positives f

LEFT JOIN bc3.padd p

    ON f.party_id = p.party_id
    AND p.data_date = f.observation_date
    AND p.addr_seq_num = 1

    AND p.data_date >= TO_DATE('2022-01-01')
    AND p.data_date < TO_DATE('2026-01-01')
),

final_state AS (

    SELECT
        party_id,
        observation_date,
        state_cde

    FROM ranked_address

    WHERE rn = 1
)

SELECT
    COALESCE(state_cde, 'MISSING') AS state_cde,

    COUNT(*) AS positive_parties,

    COUNT(*) * 1.0 /
    SUM(COUNT(*)) OVER ()
    AS pct_of_positives

FROM final_state

GROUP BY
    COALESCE(state_cde, 'MISSING')

ORDER BY
    positive_parties DESC
"
) %>%

```

collect()

positive_state_distribution

Y hay una mejora importante para el documento que te pidieron: después de obtener la distribución de `label_3m = 1`, haría **exactamente la misma extracción para `label_3m = 0`**. Así podremos presentar el verdadero análisis de balance:

State	Positive N	Positive %	Negative N	Negative %	Representation ratio
-------	---------------	------------	------------	------------	-------------------------

con:

`\[ RepresentationRatio= \frac{\% positives\ in\ state}{\% negatives\ in\ state} \]`

Eso responde directamente a la solicitud de revisar el **balance between victims/non-victims by location**, en lugar de mostrar solamente dónde viven los positivos.

Primero ejecuta `padd_primary_qa`. Con ese resultado confirmamos si `addr_seq_num = 1` por sí solo resuelve la multiplicidad o si necesitamos mantener también el desempate por `upd_tsp`.